

React 一复习要点

1 React 简介

1.1 React 的由来

[p.4]

- > 2011 年 Facebook 面临新闻动态页面复杂化挑战 [p.4]:
 - 代码复杂度急剧上升：大量 DOM 操作与业务逻辑混杂
 - 数据同步困难：多个组件共享状态时难以保持一致
 - 性能问题：频繁 DOM 更新导致页面卡顿
- > 工程师 **Jordan Walke** 创建 React，2013 年开源 [p.4]

1.2 React 的核心特点

[p.5–6]

- > **React**：一个用于构建用户界面的 JavaScript 库 (Library) [p.5]
- > 设计理念 [p.5]:
 - **Learn Once, Write Anywhere**: Web、移动端 (React Native)、桌面 (Electron)
 - **组件化**: UI 拆分为独立、可复用的组件
 - **状态驱动视图**: UI 是状态的函数 $f(state) = UI$
- > 核心特性速查 [p.6]:

特性	描述	解决的问题
声明式编程	描述 UI 状态，React 自动处理 DOM 更新	避免手动 DOM 操作
组件化	UI 拆分为可复用组件	提高复用性，便于协作
虚拟 DOM	高效 DOM 更新策略	提升性能，减少重排重绘
单向数据流	数据流向清晰可预测	便于调试和追踪状态
JSX 语法	JS 中编写类 HTML 结构	提升开发效率和可读性

1.3 库 (Library) vs 框架 (Framework)

[p.7–8]

- > React 作为库的特点 [p.7]: 只专注视图层，不强制项目结构，不提供路由/状态管理/HTTP 等完整方案

维度	库 (Library)	框架 (Framework)
控制权	开发者调用库的 API	框架控制执行流程
约束程度	低约束，灵活使用	高约束，遵循约定
关注点	解决特定问题	提供完整解决方案
扩展性	易于替换和组合	难以替换核心模块
代表	React	Angular

2 React 核心概念详解

2.1 JSX 语法

[p.10–14]

- > **JSX** (JavaScript XML) [p.11]: JavaScript 扩展语法, 允许在 JS 中编写类 HTML 结构。被 Babel 编译器转换为 JS 函数调用
- > 编程范式对比 [p.11]:
 - **命令式编程** (Imperative): 告诉计算机”怎么做”, 描述执行步骤
 - **声明式编程** (Declarative): 告诉计算机”做什么”, 描述目标状态
- > JSX 核心规则 [p.12]:
 - 必须有**根元素包裹** (只能返回一个根元素)
 - 标签必须闭合, 自闭合标签用 `</>`
 - 使用 `className` 替代 `class` (`class` 是 JS 关键字)
 - 使用 `htmlFor` 替代 `for`
 - `{ }` 大括号内写任意 JavaScript 表达式

2.2 组件化开发

[p.16–20]

- > 将复杂 UI 拆分为独立、可复用的组件 [p.16], 优势: 高复用性、低耦合、易测试、可维护、便于团队协作
- > **函数组件** [p.17]: 以大写字母开头 (PascalCase), 文件名与组件名一致
- > **Props** (Properties) [p.18–19]:
 - 组件间传递数据的方式, 从父组件流向子组件 (单向)
 - **只读性**: 子组件不能修改 props, 保证数据单向性
 - 可设置类型检查 (PropTypes/TypeScript) 和默认值
- > 子组件通过**回调函数**向父组件传递数据 [p.20]

2.3 状态管理—useState

[p.22–25]

- > 为什么需要 State [p.22]: 普通变量变化不会触发组件重新渲染, UI 无法更新
- > **useState Hook** [p.23]:
 - State 变化时 React 自动重新渲染组件 (响应式)
 - State 是组件私有的内部可变数据, 外部无法访问
 - **不可直接修改 State**, 必须通过 `setState` 函数更新
 - 多个 State 会**批量更新** (Batching), 提高性能 (异步更新)
- > 基本语法: `const [state, setState] = useState(initialValue)` [p.23]

2.4 副作用—useEffect

[p.27–31]

- > **useEffect**: 与组件渲染无关的操作, 用 Hook 实现 [p.27]
- > 常见副作用: 数据获取 (API 调用)、订阅事件 (WebSocket)、DOM 操作、定时器、日志记录 [p.27]
- > 依赖数组控制执行时机 [p.28]:

依赖数组	执行时机	适用场景
<code>[]</code> 空数组	仅在挂载时执行一次	初始化、一次性数据获取
<code>[dep]</code> 有依赖	挂载时 + 每次 dep 变化	依赖特定数据的操作
不写 (无依赖)	每次渲染都执行	需同步更新的操作

> **useState vs useEffect** [p.31]: useState 管理状态数据（存储更新）；useEffect 处理副作用（执行时机）

2.5 虚拟 DOM

[p.33–34]

- > 虚拟 DOM 是真实 DOM 的 **JavaScript 对象表示** [p.33]
- > 为什么需要 [p.33]: 直接操作真实 DOM 非常昂贵，触发重排 (Reflow) 和重绘 (Repaint)
- > 工作原理四步 [p.33]:
 1. 首次渲染: JSX → 虚拟 DOM 树 → 真实 DOM
 2. 状态更新: 创建新的虚拟 DOM 树
 3. **Diff 算法**: 高效对比新旧虚拟 DOM 树差异
 4. 批量更新: 计算最小 DOM 更新操作，批量应用到真实 DOM
- > Diff 算法特点 [p.34]: 同层比较、类型不同则替换、**key** 属性优化列表对比

2.6 最佳实践

[p.35]

- > 单一职责原则、状态提升、代码组织规范、命名规范 (PascalCase)、导入顺序规范

3 MPA 与 SPA 架构

[p.37–38]

- > **MPA** (Multi-Page Application) [p.37]: 传统架构，每次导航重新加载整个页面
- > **SPA** (Single-Page Application) [p.37]: 现代架构，一个 HTML，JS 动态更新内容
- > SPA 核心技术 [p.37]: 前端路由 (React Router)、虚拟 DOM、状态管理 (Redux/Context)

维度	MPA	SPA
页面加载方式	每次导航重新加载整页	首次后仅局部更新
用户体验	有页面刷新感	流畅无刷新
首屏加载	较快	较慢（需加载完整应用）
SEO 友好性	天然友好	需 SSR/SSG 处理
开发复杂度	较低	较高（路由、状态等）
适用场景	简单/内容型网站	复杂/交互型应用

4 本章小结与考点

[p.40]

- > **React 简介** [p.4–8]: 起源 (Facebook 2013)、核心定位（视图层库）、库 vs 框架
- > **核心概念** [p.10–35]: JSX 规则、组件化 +Props (单向数据流)、useState 状态管理、useEffect 副作用、虚拟 DOM + Diff 算法
- > **MPA vs SPA** [p.37–38]: 加载方式、SEO、首屏、适用场景对比

高频考点: (1) React 五大核心特性 [p.6]; (2) 库 (Library) vs 框架 (Framework) 的核心区别 [p.7-8]; (3) JSX 五项核心规则 [p.12]; (4) Props 的只读性和单向数据流 [p.19]; (5) useState 特性（不可直接修改、状态批量更新）[p.23]; (6) useEffect 三种依赖数组及执行时机 [p.28]; (7) 虚拟 DOM 四步工作原理和 Diff 算法 [p.33-34]; (8) MPA vs SPA 对比 [p.38]